University of Science and Technology of Hanoi (USTH)



Introduction to Cryptography

Midterm Project Report

BroadcastGuard

A Controlled Study of Low-Exponent RSA Broadcast Recovery and Authenticated Multi-Recipient Sharing

---

Authors

Nguyễn Tiến Dũng	23BA14068
Vũ Minh Châu	23BA14028
Nguyễn Minh Hiếu	23BA14105
Lê Đức Anh	23BA14005
Hoàng Lê Anh Đức	23BA14057
Lê Vân	2411065
Phạm Duy Đức Tâm	23BA14253

Hanoi, November 2026

Abstract

Sending one unpadded integer to multiple RSA recipients with public exponent $e = 3$ can expose that integer without any private key. BroadcastGuard studies the exact CRT-and-cube-root condition, compares the same attack against RSA-OAEP under paired keys, and implements authenticated file sharing for several recipients. Twenty independent RSA key-set blocks produced 560 controlled subset observations, all consistent with the predicted integer-recovery condition. In 60 same-key comparisons, the tested attack recovered the textbook RSA message in 60 cases and the OAEP message in none; intended OAEP recipients decrypted in all 60. For a 1 MiB hybrid-encrypted file and one recipient, JSON/Base64 serialization accounted for 93.78% of the measured encryption workflow mean on one Windows machine. Eleven selected package-validity cases matched expected outcomes. The contribution is a reproducible educational integration and evidence-based analysis of established cryptographic methods, not a new primitive or a proof of universal security.

TABLE OF CONTENTS

1 Introduction and research questions **1**
2 Cryptographic background and threat model **2**
3 BroadcastGuard system design **2**
4 Experimental design and evidence **3**
5 Results and analysis **4**
5.1 E1: the integer boundary determines broadcast recovery 4
5.2 E2: paired textbook RSA and OAEP comparison 5
5.3 E3: cryptographic work, serialization and package size 5
5.4 E4: selected failure and integrity cases 6
6 Critical discussion and limitations **7**
7 Conclusion **7**
References **7**

1 Introduction and research questions

RSA public encryption transforms an integer under a public modulus and exponent [1]. If the same integer is encrypted without padding under distinct moduli with a small exponent, the ciphertexts retain an algebraic relationship accessible to an observer. Håstad examined broader systems of low-degree modular equations [2]; the fixed-message case studied here has a direct Chinese Remainder Theorem (CRT) derivation. This deliberately unsafe example exposes why correct arithmetic and correct security assumptions must be considered together. It does not imply that a properly configured RSA-OAEP deployment is vulnerable to the same simple procedure.

A demonstration that Eve sometimes recovers a message is incomplete without specifying *when* recovery is mathematically possible, what Eve actually learns, and how the outcome changes when the encryption scheme is replaced. It also leaves unanswered how to protect a file too large for direct RSA-OAEP encryption. BroadcastGuard connects these questions through a public-only attack engine, a seven-subset conditions analyzer, a same-key OAEP comparison, and an AES-GCM/RSA-OAEP multi-recipient package. The interactive application illustrates the algorithms; its interface is not independent experimental evidence.

The study addresses four questions: **RQ1**, which numerical condition characterizes recovery of a shared integer by CRT-and-root? **RQ2**, what changes when only textbook RSA is replaced by randomized RSA-OAEP while keys, plaintext and attacker procedure remain fixed? **RQ3**, how do file size, recipient count and JSON/Base64 packaging affect hybrid-encryption time and storage? **RQ4**, which selected invalid or altered packages are rejected without returning plaintext?

2 Cryptographic background and threat model

In the educational attack, Alice, Bob and Charlie have independent 2048-bit RSA moduli n_i and the common public exponent $e = 3$. A nonempty byte string is interpreted as a nonnegative big-endian integer M, checked to satisfy $M < n_i$ for each recipient, and encrypted as $c_i = M^3 \bmod n_i$. Eve sees the public (n_i, e, c_i) observations but not the private keys or original message. The moduli are required to be pairwise coprime; the builder checks this property rather than silently assuming it.

For any nonempty observed subset S, let $N_S = \prod_{i \in S} n_i$. CRT gives a unique residue $X \in [0, N_S)$ satisfying $X \equiv c_i \pmod{n_i}$ for every $i \in S$. More explicitly, with $N_i = N_S/n_i$ and $u_i = N_i^{-1} \bmod n_i$,

$$X = \left(\sum_{i \in S} c_i N_i u_i \right) \bmod N_S = M^3 \bmod N_S. \tag{1}$$

Pairwise coprimality ensures $\gcd(N_i, n_i) = 1$, so each inverse exists. The attack computes $r = \lfloor \sqrt[3]{X} \rfloor$ by integer arithmetic and checks $r^3 = X$; floating-point roots are unsuitable for RSA-sized integers. If $M^3 < N_S$, then $X = M^3$ and $r = M$. Conversely, under the stated valid-input assumptions this particular nonnegative CRT-and-root procedure returns M only if $M^3 < N_S$. The weaker condition $M < N_S$ is insufficient. A perfect cube may yield an exact but *wrong* candidate; Eve cannot test equality with an unknown original M. A separate experiment evaluator performs that comparison and checks the inequality.

Integer recovery must not be confused with byte-for-byte recovery. Big-endian conversion maps both the single byte `0x02` and the 96-byte sequence $\texttt{0x00}^{95} \| \texttt{0x02}$ to $M = 2$. In E1, Eve can recover integer 2 from one ciphertext, but cannot infer that the original serialization had 95 leading zero bytes. The evaluator knows the supplied length and uses it to reconstruct the original bytes. Accordingly, the E1 recovery indicator means *evaluator-confirmed integer equality*; exact original bytes require a length or encoding convention known independently.

RSA-OAEP changes the integer representative before RSA exponentiation through randomized encoding [3, 4]. The simple shared unpadded-M^3 relation therefore does not describe OAEP ciphertexts. The controlled defense experiment uses RSAES-OAEP with SHA-256, MGF1(SHA-256) and the empty label. RFC 8017 specifies $m_{\text{Len}} \leq k - 2h_{\text{Len}} - 2$; for a 256-byte RSA-2048 modulus and a 32-byte SHA-256 output, this limits input to 190 bytes [4]. Observing distinct OAEP ciphertexts illustrates randomization but is not, by itself, a formal security proof.

3 BroadcastGuard system design

Figure 1 shows the three computational paths and the separate evidence pipeline. The educational attack modules deliberately use textbook RSA; secure file sharing uses standard authenticated encryption and never calls the textbook operation. This separation prevents a vulnerable research primitive from becoming a file-encryption dependency.

The *scenario builder* converts input bytes to M, generates three RSA-2048/$e = 3$ keypairs, checks $M < n_i$ and pairwise coprimality, and computes $c_i = \text{pow}(M, 3, n_i)$. Only public numbers survive in its observations. The *attacker engine* accepts selected public observations, rejects duplicate or incompatible inputs, performs the explicit CRT computation in Eq. (1), and computes an integer cube root with an exactness flag. It is not given plaintext or a private key. The *experiment evaluator* separately knows M and reports whether a candidate matches it. The *conditions analyzer* invokes these components for all seven nonempty subsets, including non-prefix pairs such as Bob–Charlie. It reports both the exact-root status and the evaluator-confirmed integer match, with the minimum successful interception count or an explicit no-success result.

The *defense lab* uses the same three RSA-2048/$e = 3$ keypairs and plaintext for textbook encryption and RSA-OAEP. It applies the same public CRT-and-root procedure to both sets of ciphertext integers; intended recipients also decrypt their OAEP ciphertexts with in-memory private keys. Holding the keys fixed isolates

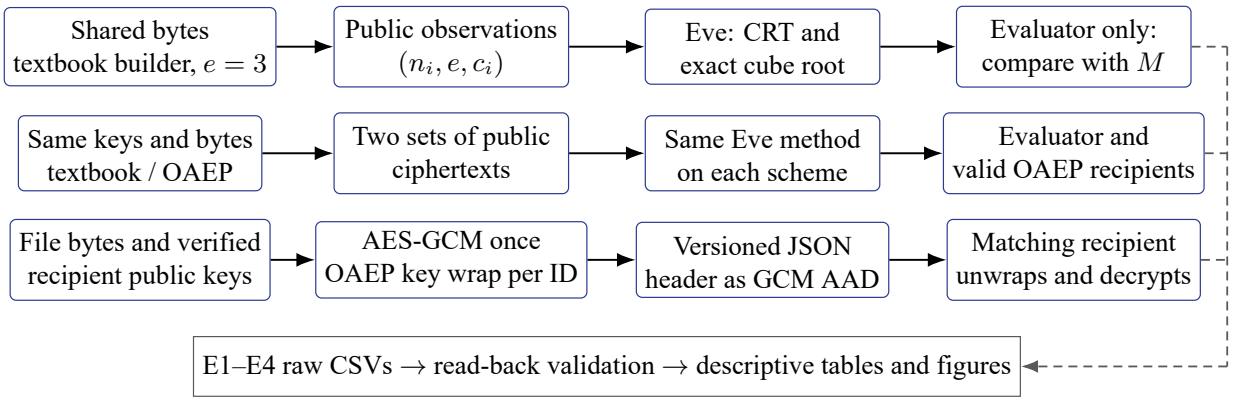


Figure 1: Implemented data flows. Eve receives only public observations; original plaintext reaches the evaluator, not the attack engine. Defense compares two schemes under the same RSA keys. The hybrid path encrypts the file once and wraps its fresh AES key separately for each recipient. The evidence pipeline validates archived experiment records before analysis.

the scheme change within each paired trial. OAEP ciphertext integers are tested as attack inputs, but they are never mislabeled as textbook ciphertexts. Repeating OAEP encryption under one key gives a separate observation of ciphertext diversity.

The *secure sharing* module uses RSA-2048/$e = 65537$, not the educational $e = 3$ configuration. Each file encryption creates a fresh 256-bit AES key and 96-bit GCM nonce. AES-GCM encrypts the file once; RSA-OAEP with SHA-256/MGF1 wraps the AES key independently for each of one to five authenticated recipient public keys. A canonical UTF-8 JSON header records format and algorithm identifiers, filename, original size, nonce, recipient IDs, public-key fingerprints and wrapped keys. Its serialized bytes serve as GCM associated authenticated data (AAD). The package contains Base64-encoded ciphertext with the 16-byte tag and no private or plaintext key material. The receiver validates structure and key fingerprint, unwraps its entry, reconstructs AAD and returns plaintext only after authenticated decryption and size verification [5]. Fingerprints label keys; they do not independently prove a person's identity.

The *evaluation pipeline* stores individual E1–E4 records, a configuration manifest and explicit validation status. A read-back validator checks case coverage, paired-key consistency, timing identities and outcome invariants before statistical analysis. Figure generation reads the committed official CSVs; it does not rerun randomized cryptographic experiments. Source code, the run and reproduction instructions are available in the repository.

4 Experimental design and evidence

The four experiments count a result only after its correctness invariant is checked. E1 and E2 use 20 independently generated RSA key-set blocks; the three E2 messages within each block are paired observations, not independent key sets. Cryptographic randomness was not seeded; the recorded schedule seed controls workload order only.

E1's four messages are `B` repeated 1, 96 or 180 times, and 95 zero bytes followed by `0x02`; the two 96-byte samples test the distinction between byte count and numeric value. Each of the 20 blocks evaluates all seven subsets for every sample, including every pair. E2 reuses the block's keys in both schemes, using the 1-, 96- and 180-byte `B` samples. Textbook and OAEP ciphertexts are each submitted to the same attacker implementation. OAEP encryption is additionally repeated under one public key and every intended recipient's decryption is checked. This design can attribute a within-pair difference to the scheme change more clearly than a comparison using independently generated keys, while retaining the limitation that 60 pairs are clustered within 20 blocks.

E3 prepares a five-recipient public-key set before routine timing, uses its first 1, 3 or 5 members and

encrypts 10 KiB, 100 KiB and 1 MiB synthetic files with a *fresh* AES key/nonce on every repetition. Two warmups per configuration are excluded; 20 measured repetitions remain for each of nine configurations. The monotonic `perf_counter_ns` interval `encryption_ns` spans key generation for AES, RSA-OAEP key wrapping, AAD creation and AES-GCM content encryption. `serialization_ns` spans canonical JSON/Base64 output; `total_encryption_ns` is their sum. RSA recipient-key generation is measured separately in 60 records, so it is not silently included in routine encryption. `decryption_ns` measures the first valid recipient only. Every decrypted byte string is compared with the input, and every serialized size is measured from actual package bytes. Reported summaries use all individual timings, with means, medians, sample standard deviations and quartiles; no outlier was removed.

The official manifest records Windows 11, Python 3.12.10, `cryptography` 48.0.1, an Intel64 Family 6 Model 191 CPU with 20 logical CPUs, run ID `official-20261106T083221Z-3bbf57cc`, and source commit `46c4558`. Hardware-specific timing must not be generalized to cloud deployments. E4 checks one valid package and one mutation for each of ten rejection conditions; it records both the observed category and whether any plaintext escaped. The archived official evidence and reproduction procedure support read-back validation without repeating RSA key generation or benchmarks.

5 Results and analysis

5.1 E1: the integer boundary determines broadcast recovery

Figure 2 shows all 28 sample–subset combinations across the 20 key-set blocks. Every one of the 560 rows agreed with the prediction $M^3 < N_S$. The minimum number of intercepted ciphertexts was one for $\mathtt{B}\times 1$, two for $\mathtt{B}\times 96$, three for $\mathtt{B}\times 180$, and one for the leading-zero 96-byte sample. For these keys, all three two-recipient choices recover the $\mathtt{B}\times 96$ integer, whereas only the full three-recipient set recovers $\mathtt{B}\times 180$. Selection matters through the product N_S, rather than through a fixed positional rule. The very short message can be recovered from one ciphertext because its cube already lies below one 2048-bit modulus.

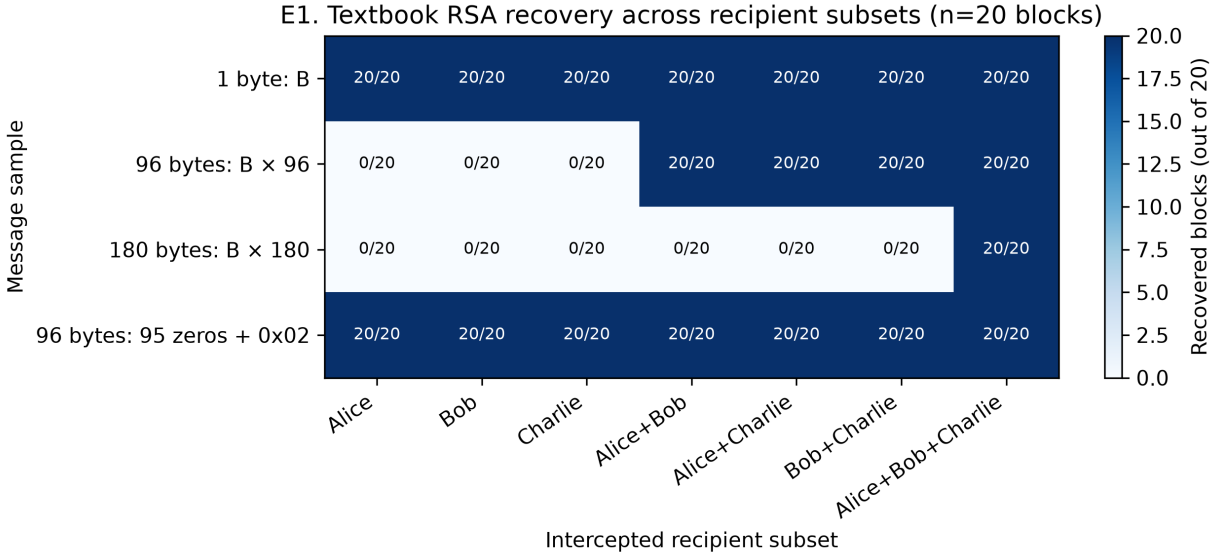


Figure 2: Evaluator-confirmed *integer* recovery for four fixed textbook RSA messages and all seven nonempty recipient subsets; each cell is successful key-set blocks out of 20. The two 96-byte rows encode different integers. The leading-zero row's original 96-byte serialization is not inferred by Eve without independent length information.

Although both samples occupy 96 bytes, $\mathtt{B}\times 96$ is a 767-bit integer and $\mathtt{0x00}^{95}\|\mathtt{0x02}$ equals 2. Eve can recover 2 from one ciphertext but cannot infer its original byte length; the plotted success metric is evaluator-confirmed integer equality.

5.2 E2: paired textbook RSA and OAEP comparison

Table 1 summarizes the 60 paired tests. Each pair holds plaintext, three RSA keypairs, $e = 3$, recipient set and CRT-and-root attacker fixed. Textbook RSA presented the repeated integer relation, and the tested procedure returned the original integer in every pair. OAEP randomized the encoded representatives before RSA exponentiation; the same procedure recovered the original plaintext in no paired test. Legitimate OAEP decryption succeeded in every pair, and repeated OAEP encryptions under the same key yielded distinct ciphertexts in all 60 checks.

Table 1: E2 paired observations across 20 independent RSA key-set blocks and three messages per block. "Attack recovery" means evaluator-confirmed equality with the original message; legitimate decryption is a different operation.

Scheme	Evaluated operation	Correct / 60 pairs
Textbook RSA	Eve's CRT-and-root recovery	60/60
RSA-OAEP	Eve's same CRT-and-root recovery	0/60
RSA-OAEP	Intended recipients' decryption	60/60

Using the same keys prevents a changing modulus set from explaining the within-pair contrast. The observations do not prove RSA-OAEP secure against every attack, nor do 60 comparisons represent 60 independent key sets. They establish a narrower result: the controlled CRT-and-root procedure that exploits a shared unpadded M did not recover the OAEP plaintext in these trials. OAEP's randomized encoding is specified by RFC 8017 [4]; repeated-ciphertext diversity is illustrative evidence of that behavior, not a security reduction.

5.3 E3: cryptographic work, serialization and package size

Figure 3 separates *encrypt+wrap* from JSON/Base64 serialization. For a 1 MiB file and one recipient, the 20 raw measurements have means of 0.479550 ms and 7.232030 ms, respectively; their 7.711580 ms total excludes RSA recipient-key generation. Serialization accounts for 93.78% of that observed mean. This bottleneck belongs to the chosen Python package representation and workflow, not to AES-GCM itself. The figure uses independent zero-based vertical scales for its three file-size panels; bar heights must be compared within panels or by reading their numerical axes.

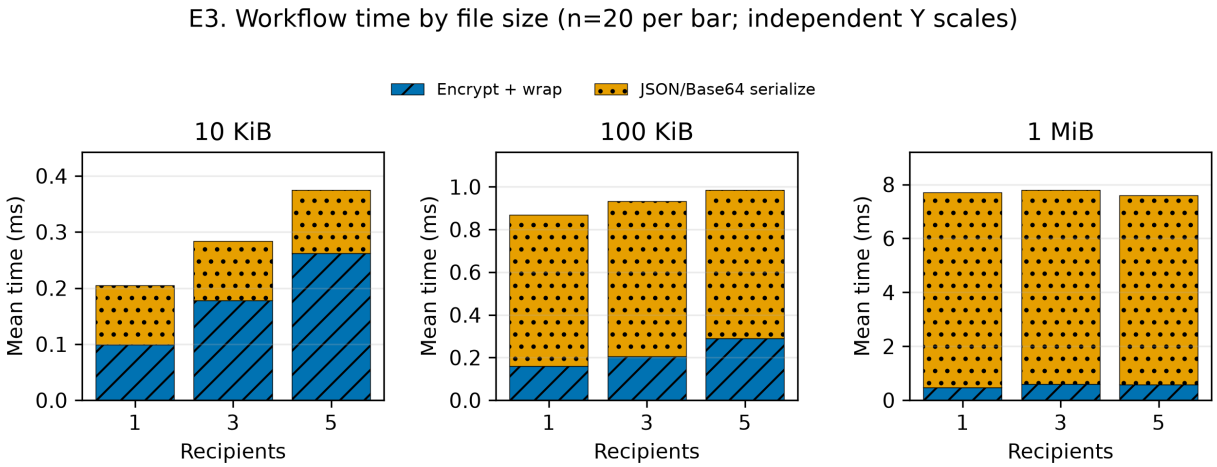


Figure 3: Mean E3 encrypt+wrap and JSON/Base64 serialization durations in milliseconds, 20 timed repetitions per file-size/recipient configuration using prepared RSA keys. Panels have independently labeled, zero-based Y scales. Routine total encryption is the sum of these two components; recipient-key generation was timed separately.

Table 2 also reports medians and interquartile ranges (IQRs).

Table 2: E3 total encryption workflow: descriptive timing statistics and serialized-package overhead. Each row has 20 timed repetitions; times are milliseconds. IQR is the recorded 75th minus 25th percentile. Key generation is excluded.

File	Recipients	Mean	Median	IQR	Overhead (%)
10 KiB	1	0.205	0.197	0.088	40.28
	3	0.283	0.293	0.084	49.40
	5	0.375	0.354	0.097	58.52
100 KiB	1	0.870	0.848	0.135	34.03
	3	0.934	0.867	0.132	34.94
	5	0.985	0.960	0.106	35.85
1 MiB	1	7.712	7.401	0.406	33.40
	3	7.795	7.548	0.263	33.49
	5	7.594	7.521	0.128	33.58

The three 1 MiB total means are 7.712, 7.795 and 7.594 ms for one, three and five recipients. Their non-monotonic order does not establish a speed advantage for five recipients: the differences are small compared with measured variation and may reflect scheduling or interpreter effects. The decryption interval covers only the first valid recipient, not sequential decryption for all recipients.

The actual serialized package follows an accounting identity for the current canonical JSON format. If L is plaintext length, r is recipient count, f is filename and $\mathcal{I}$ denotes recipient identifiers, then

$$P(L, r) = 4 \left\lceil \frac{L + 16}{3} \right\rceil + H(L, r, f, \mathcal{I}). \tag{2}$$

The first term is Base64 encoding of the L encrypted content bytes plus the 16-byte GCM tag. H covers JSON delimiters, algorithm/version labels, decimal file size, nonce, filename, fingerprints and a Base64-encoded 256-byte RSA-2048 wrapped AES key for each recipient. It is not constant for arbitrary filenames and identifiers. For the fixed E3 metadata the model matched every measured package size with zero residual because it reconstructs the serializer's exact structure; this is an accounting check, not an independently fitted law or a cryptographic proof. At 10 KiB, observed overhead rises from 40.28% with one recipient to 58.52% with five. At 1 MiB it is 33.40% to 33.58%: Base64's approximately one-third expansion dominates while per-recipient metadata becomes relatively smaller. AES-GCM's tag alone would add far less.

5.4 E4: selected failure and integrity cases

All 11 correctness cases matched their expected status. The valid package with a matching recipient key returned the original bytes. Ten invalid, mismatched or modified inputs returned no plaintext: wrong private key, absent recipient ID, changed encrypted content, changed GCM tag, changed authenticated filename, changed nonce, changed wrapped key, shortened recipient list, unsupported version and malformed JSON. The observed API classified the last two as structural `invalid_package` and the other eight as `decryption_failed`. This classification does not claim to identify malicious intent or the exact cryptographic cause of every failure. The tests illustrate package validation and fail-closed behavior for selected inputs; they are not an estimate of a universal attack-rejection probability.

6 Critical discussion and limitations

E1's agreement with the exact $M^3 < N_S$ prediction depends on a common integer, valid textbook RSA inputs and pairwise-coprime moduli. The leading-zero case exposes a distinct encoding limit: modular arithmetic recovers an integer, while byte-for-byte reconstruction needs a known length or convention. Showing the original 96 bytes in a demonstration therefore requires disclosure that the evaluator supplied their length.

E2 holds keys and plaintext fixed within each pair, isolating the scheme change more clearly than independently generated keys would. Yet 0/60 OAEP recoveries concern only the tested CRT-and-root procedure and three messages. The 60 comparisons cluster within 20 independent key-set blocks; neither they nor OAEP ciphertext diversity constitute a formal security proof. The implementation uses library OAEP rather than designing new padding.

E3 separates cryptographic work from packaging. A more compact representation is a sensible future comparison, but was not measured. JSON/Base64 trades storage and serialization time for inspectability. Twenty repetitions per configuration on one Windows computer do not predict performance on other hardware or in the cloud. Recipient-key generation was timed separately, and decryption timing covers only one recipient.

Secure sharing assumes recipient public keys were authenticated before encryption; a fingerprint identifies a key but not a person. AES-GCM authenticates ciphertext and AAD under the unwrapped key [5], not the sender. The package leaves filename and recipient metadata visible and has no sender signature, replay protection, PKI, revocation or persistent private-key management. E4's selected rejections cannot cover all malformed inputs or establish intent. Such extensions need an explicit threat model and new evidence.

7 Conclusion

BroadcastGuard connects the exact textbook RSA broadcast condition to a same-key OAEP comparison and authenticated multi-recipient file sharing. Its 560 E1 observations verify the known integer-recovery bound; 60 E2 pairs show non-recovery by the selected attack against OAEP alongside valid decryption. E3 quantifies the JSON/Base64 packaging cost, and E4 verifies selected fail-closed cases. The contribution is controlled integration and interpretation of established methods, not a new primitive or a universal security proof.

References

[1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978, doi:10.1145/359340.359342.

[2] J. Hastad, "Solving simultaneous modular equations of low degree," *SIAM Journal on Computing*, vol. 17, no. 2, pp. 336–341, 1988, doi:10.1137/0217019.

[3] M. Bellare and P. Rogaway, "Optimal asymmetric encryption," in *Advances in Cryptology – EUROCRYPT '94*, ser. Lecture Notes in Computer Science, vol. 950. Springer, 1995, pp. 92–111, doi:10.1007/BFb0053428.

[4] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," RFC Editor, RFC 8017, 2016. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8017.html

[5] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/counter mode (GCM) and GMAC," National Institute of Standards and Technology, Special Publication 800-38D, 2007, doi:10.6028/NIST.SP.800-38D.